

알고리즘 학습용 정리

DP 모든 것 정리

동적 계획법(DP, Dynamic Programming)을 처음 배우는 기준으로 한 번에 볼 수 있게 정리한 문서입니다. 개념, 푸는 순서, 대표 유형, 자주 하는 실수, 예제 코드까지 담았습니다.

1. DP를 한 줄로 말하면

DP는 큰 문제를 작은 문제로 나누고, 그 작은 문제의 답을 저장해 두면서 재사용하는 방법입니다.

핵심 감각: 같은 계산을 여러 번 하지 않게 만드는 것

2. 왜 DP가 필요한가

- 재귀만 쓰면 같은 문제를 반복해서 계산하는 경우가 많습니다.
- 한 번 구한 답을 저장해 두면 속도를 크게 줄일 수 있습니다.
- 특히 최댓값, 최솟값, 경우의 수 문제에 자주 나옵니다.

3. DP가 성립하는 조건

3-1. 최적 부분 구조

큰 문제의 답이 작은 문제의 답들로부터 만들어질 수 있어야 합니다.

3-2. 중복되는 부분 문제

같은 작은 문제를 여러 번 다시 계산하는 구조여야 합니다.

4. 가장 쉬운 예시: 피보나치

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

이 코드는 `fib(2)`, `fib(3)` 같은 값을 여러 번 다시 계산합니다. DP는 이런 낭비를 막습니다.

```
def fib(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]
```

5. DP 구현 방식 두 가지

방식	설명	장점	단점
Top-Down	재귀로 내려가며 필요할 때 계산하고 저장	점화식을 그대로 쓰기 쉬움	재귀 깊이, 함수 호출 오버헤드
Bottom-Up	작은 문제부터 반복문으로 차례대로 계산	안정적이고 실행 흐름이 명확함	처음엔 채우는 순서가 덜 직관적일 수 있음

5-1. 메모이제이션

Top-Down에서 계산한 값을 저장하는 방식입니다.

5-2. 타블레이션

Bottom-Up에서 표를 채우면서 계산하는 방식입니다.

6. DP 문제를 푸는 표준 순서

1. **dp 정의**: `dp[i]` 가 무엇을 뜻하는지 정합니다.
2. **점화식**: 현재 상태를 어떤 이전 상태에서부터 만들지 정합니다.
3. **초기값**: 시작 상태를 정합니다.
4. **계산 순서**: 어떤 순서로 값을 채워야 하는지 정합니다.

DP는 구현보다 먼저 **dp 정의**가 제일 중요합니다.

7. dp 정의가 왜 중요한가

DP 문제의 절반 이상은 `dp[i]` 를 정확히 정하는 데서 걸립니다.

- `dp[i]` = i 번째 계단에 도착하는 방법 수
- `dp[i]` = i 를 1로 만드는 최소 연산 횟수
- `dp[i][j]` = i 번째 물건까지 고려했을 때 무게 j 이하의 최대 가치

8. 대표 예제 1: 계단 오르기

한 번에 1칸 또는 2칸 올라갈 수 있을 때, n 칸에 도착하는 방법 수를 구합니다.

정의

$dp[i]$ = i 칸에 도착하는 방법 수

점화식

$dp[i] = dp[i-1] + dp[i-2]$

초기값

$dp[1] = 1$, $dp[2] = 2$

```
def stairs(n):
    if n == 1:
        return 1

    dp = [0] * (n + 1)
    dp[1] = 1
    dp[2] = 2

    for i in range(3, n + 1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]
```

9. 대표 예제 2: 1로 만들기

정수 x 가 있을 때 -1 , $/2$, $/3$ 연산으로 1을 만드는 최소 횟수를 구합니다.

$dp[i]$ = i 를 1로 만드는 최소 연산 횟수

```
def make_one(x):
    dp = [0] * (x + 1)

    for i in range(2, x + 1):
        dp[i] = dp[i - 1] + 1

        if i % 2 == 0:
            dp[i] = min(dp[i], dp[i // 2] + 1)
        if i % 3 == 0:
            dp[i] = min(dp[i], dp[i // 3] + 1)

    return dp[x]
```

10. 대표 예제 3: 배낭 문제

각 물건에 무게와 가치가 있을 때, 제한 무게를 넘지 않으면서 가치 합을 최대 만드는 문제입니다.

$dp[i][w]$ = i 번째 물건까지 고려했을 때, 무게 w 이하의 최대 가치

핵심은 항상 두 가지 선택입니다.

- 현재 물건을 담지 않는다

- 현재 물건을 담는다

11. 대표적인 DP 유형

경우의 수 DP

몇 가지 방법이 있는지 세는 문제

최솟값/최댓값 DP

최소 비용, 최대 점수, 최대 가치

문자열 DP

LCS, 편집 거리 같은 문자열 문제

배낭형 DP

선택/비선택 구조가 있는 누적 최적화 문제

구간 DP

`dp[i][j]` 처럼 구간 단위로 푸는 문제

트리/비트마스크 DP

상태 관리가 더 복잡한 확장형 문제

12. 1차원 DP와 2차원 DP

1차원 DP

`dp[i]` 형태로 표현합니다. 계단 오르기, 피보나치 같은 문제에서 자주 씁니다.

2차원 DP

`dp[i][j]` 형태로 표현합니다. 배낭, LCS처럼 상태가 2개 필요할 때 자주 씁니다.

13. DP와 다른 기법 비교

기법	핵심	DP와 차이
분할정복	문제를 쪼개서 각각 푼 뒤 합침	작은 문제들이 보통 서로 안 겹침
그리디	지금 당장 최선으로 보이는 선택	미래 전체 최적을 항상 보장하지 않음
DP	작은 문제 답을 저장하고 재사용	중복 부분 문제와 점화식이 중요

14. DP에서 자주 하는 실수

- `dp[i]` 정의가 애매함
- 점화식을 틀리게 세움
- 초기값을 빼먹음
- 인덱스를 헷갈림
- 계산 순서를 잘못 정함
- 정답 위치를 틀리게 출력함
- 메모리를 너무 크게 잡음

15. 공간 최적화

어떤 문제는 전체 배열이 필요 없고 바로 이전 몇 개 상태만 필요합니다. 이때는 배열 대신 변수만 써서 메모리를 줄일 수 있습니다.

```
def fib(n):
    if n <= 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

16. DP 문제를 보면 먼저 할 질문

1. 최솟값, 최댓값, 경우의 수 중 무엇을 구하는가?
2. 현재 상태를 무엇으로 표현할까?
3. 이전 어떤 상태에서 현재 상태로 올 수 있나?
4. 중복 계산이 생기나?
5. 배열 차원은 몇 개인가?

17. 입문 연습 순서 추천

1. 피보나치
2. 계단 오르기

3. 1로 만들기
4. 동전 문제
5. 배낭 문제
6. LIS
7. LCS

DP는 처음부터 어려운 문제를 붙잡기보다, **쉬운 점화식 문제로 감각을 먼저 만들고 올라가는 게 좋습니다.**

18. DP 풀이 템플릿

```
# 1. dp 정의
# dp[i] = ...

# 2. 초기값
dp[0] = ...
dp[1] = ...

# 3. 점화식
for i in range(...):
    dp[i] = ...

# 4. 정답 출력
print(dp[...])
```

19. 재혁님 기준 핵심 압축

- DP는 큰 문제를 작은 문제로 나누고, 작은 문제 답을 저장해 재사용하는 방법입니다.
- 핵심은 **dp 정의, 점화식, 초기값, 계산 순서**입니다.
- DP가 어려운 이유는 구현보다 **상태 정의**가 어렵기 때문입니다.
- 문제를 보면 먼저 "작은 문제는 뭘까?"를 생각하는 습관이 중요합니다.

20. 마지막 한 줄 정리

DP는 **작은 문제의 답을 저장해서 큰 문제를 빠르게 푸는 방법**입니다.